

OBSERVATION TASK 2

Title

Bank Account Management System using Classes and Objects

Course Outcome: CO1

Objective / Aim

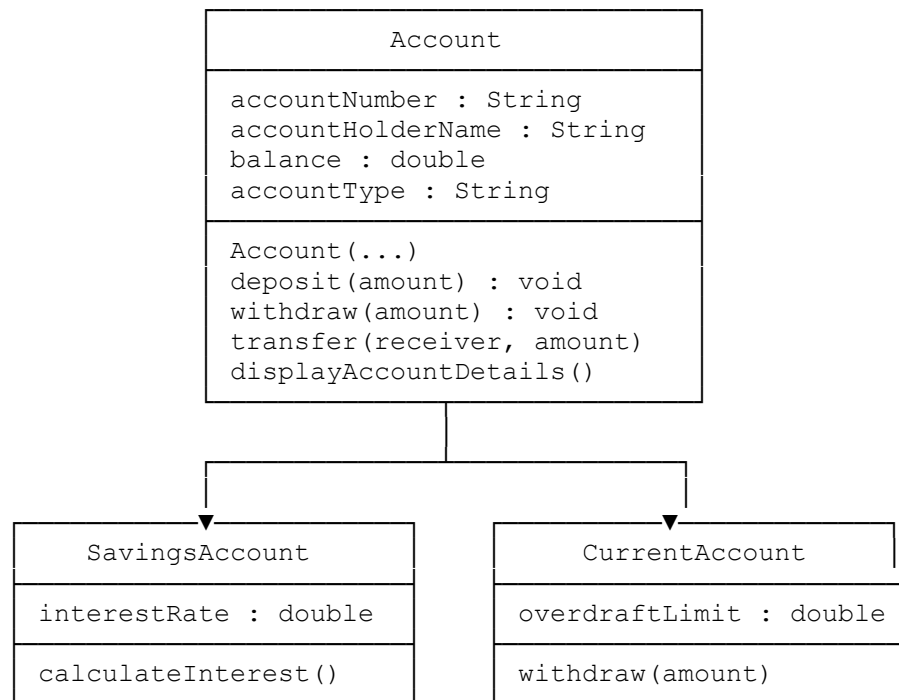
To design and implement a **Bank Account Management System in Java** using classes and objects, demonstrating object-oriented programming concepts such as **attributes, methods, constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, and fund transfers.**

Algorithm / Logic Description

1. Create a base class named `Account`.
2. Define the following attributes in the `Account` class:
 - Account number
 - Account holder name
 - Balance
 - Account type
3. Define a constructor to initialize the account details.
4. Define the following methods in the `Account` class:
 - `deposit()` to add money to the account.
 - `withdraw()` to withdraw money when sufficient balance is available.
 - `transfer()` to transfer money from one account to another.
 - `displayAccountDetails()` to display account information.
5. Create a `SavingsAccount` class that inherits from the `Account` class.
6. Add `interestRate` as an additional attribute in `SavingsAccount`.
7. Define `calculateInterest()` to calculate interest based on the current account balance and add the calculated interest to the balance.
8. Create a `CurrentAccount` class that inherits from the `Account` class.
9. Add `overdraftLimit` as an additional attribute in `CurrentAccount`.
10. Override the `withdraw()` method in `CurrentAccount` to allow withdrawals up to the permitted overdraft limit.
11. Create objects for `SavingsAccount` and `CurrentAccount`.

12. Display the account details before performing transactions.
13. Perform deposit operations on both accounts.
14. Perform withdrawal operations on both accounts.
15. Calculate and add interest to the savings account.
16. Transfer money from the savings account to the current account.
17. Display the account details after all transactions.
18. Display the final balance of both accounts.

UML Class Diagram



Inheritance Relationship

Classes Used

Account

- Stores common account information.
- Performs deposit, withdrawal, transfer, and display operations.

SavingsAccount

- Inherits from `Account`.
- Adds `interestRate`.
- Calculates and adds interest to the account balance.

CurrentAccount

- Inherits from `Account`.
- Adds `overdraftLimit`.
- Overrides the `withdraw()` method to permit overdraft withdrawals.

BankAccountDemo

- Contains the `main()` method.
- Creates account objects and performs all transactions.

Source Code

File 1: `Account.java`

```
package tasks;

public class Account {
    String accountNumber;
    String accountHolderName;
    double balance;
    String accountType;

    // Constructor
    Account(String accountNumber, String accountHolderName,
            double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
        this.accountType = accountType;
    }

    // Deposit
    void deposit(double amount) {
        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    // Withdraw
    void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance.");
        }
    }
}
```

```

// Transfer
void transfer(Account receiver, double amount) {
    if (amount > 0 && amount <= balance) {
        balance = balance - amount;
        receiver.balance = receiver.balance + amount;

        System.out.println("Transferred: " + amount);
        System.out.println("From: " + accountNumber);
        System.out.println("To: " + receiver.accountNumber);
    } else {
        System.out.println("Transfer failed.");
    }
}

// Display account details
void displayAccountDetails() {
    System.out.println("Account Number: " + accountNumber);
    System.out.println("Account Holder: " + accountHolderName);
    System.out.println("Account Type: " + accountType);
    System.out.println("Balance: " + balance);
}

}

// Savings Account
class SavingsAccount extends Account {
    double interestRate;

    SavingsAccount(String accountNumber, String accountHolderName,
                    double balance, double interestRate) {
        super(accountNumber, accountHolderName, balance, "Savings");
        this.interestRate = interestRate;
    }

    // Calculate interest
    void calculateInterest() {
        double interest = balance * interestRate / 100;
        balance = balance + interest;

        System.out.println("Interest Rate: " + interestRate + "%");
        System.out.println("Interest Added: " + interest);
    }
}

// Current Account
class CurrentAccount extends Account {
    double overdraftLimit;

    CurrentAccount(String accountNumber, String accountHolderName,
                    double balance, double overdraftLimit) {
        super(accountNumber, accountHolderName, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    // Method overriding
    @Override
    void withdraw(double amount) {

```

```

        if (amount > 0 && amount <= balance + overdraftLimit) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Withdrawal exceeds overdraft limit.");
        }
    }
}

```

File 2: BankAccountDemo.java

```

package tasks;

public class BankAccountDemo {
    public static void main(String[] args) {

        // Creating objects
        SavingsAccount savings = new SavingsAccount(
            "S101", "Raji", 10000, 5
        );

        CurrentAccount current = new CurrentAccount(
            "C101", "Akshaya", 5000, 3000
        );

        // Before transactions
        System.out.println("===== BEFORE TRANSACTIONS =====");

        System.out.println("\nSavings Account:");
        savings.displayAccountDetails();

        System.out.println("\nCurrent Account:");
        current.displayAccountDetails();

        // Deposits
        System.out.println("\n===== DEPOSITS =====");
        savings.deposit(2000);
        current.deposit(3000);

        // Withdrawals
        System.out.println("\n===== WITHDRAWALS =====");
        savings.withdraw(1000);
        current.withdraw(7000);

        // Interest
        System.out.println("\n===== INTEREST CALCULATION =====");
        savings.calculateInterest();

        // Transfer
        System.out.println("\n===== FUND TRANSFER =====");
        savings.transfer(current, 2000);

        // After transactions
        System.out.println("\n===== AFTER TRANSACTIONS =====");

        System.out.println("\nSavings Account:");
        savings.displayAccountDetails();
    }
}

```

```
        System.out.println("\nCurrent Account:");
        current.displayAccountDetails();

        // Final balances
        System.out.println("\n===== FINAL BALANCES =====");
        System.out.println("Savings Account Balance: " + savings.balance);
        System.out.println("Current Account Balance: " + current.balance);
    }
}
```

Transaction Calculation

Savings Account

Initial balance:

Rs.10000

After deposit:

$10000 + 2000 = \text{Rs.}12000$

After withdrawal:

$12000 - 1000 = \text{Rs.}11000$

Interest at 5%:

$11000 \times 5 / 100 = \text{Rs.}550$

Balance after interest:

$11000 + 550 = \text{Rs.}11550$

After transferring Rs.2000:

$11550 - 2000 = \text{Rs.}9550$

Current Account

Initial balance:

Rs.5000

After deposit:

$5000 + 3000 = \text{Rs.}8000$

After withdrawal:

$8000 - 7000 = \text{Rs.}1000$

After receiving Rs.2000 transfer:

$1000 + 2000 = \text{Rs.}3000$

Therefore, the final balances are:

Savings Account = Rs.9550

Current Account = Rs.3000

Sample Output

===== BEFORE TRANSACTIONS =====

Savings Account:
Account Number: S101
Account Holder: Raji
Account Type: Savings
Balance: 10000.0

Current Account:
Account Number: C101
Account Holder: Akshaya
Account Type: Current
Balance: 5000.0

===== DEPOSITS =====

Deposited: 2000.0

Deposited: 3000.0

===== WITHDRAWALS =====

Withdrawn: 1000.0

Withdrawn: 7000.0

===== INTEREST CALCULATION =====

Interest Rate: 5.0%

Interest Added: 550.0

===== FUND TRANSFER =====

Transferred: 2000.0

From: S101

To: C101

===== AFTER TRANSACTIONS =====

Savings Account:
Account Number: S101
Account Holder: Raji
Account Type: Savings
Balance: 9550.0

```
Current Account:
Account Number: C101
Account Holder: Akshaya
Account Type: Current
Balance: 3000.0

===== FINAL BALANCES =====
Savings Account Balance: 9550.0
Current Account Balance: 3000.0
```

Result

The **Bank Account Management System** was successfully implemented in Java using **classes and objects**. The program demonstrates **constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, fund transfers, and balance management**. The final balance of the Savings Account is **Rs.9550**, and the final balance of the Current Account is **Rs.3000**.